# AI Agent 전문 개발 과정

## Day 2 — 제어 흐름 & 상태 관리

LangChain · LangGraph · Deep Agents 프레임워크로 배우는 Agent 설계

5일 과정 (40시간) · 강의 30% / 실습 70%

# 5일 커리큘럼

| Day 1 | Day 2 | Day 3 | Day 4 | Day 5 |
|---|---|---|---|---|
| 문제 정의 & 설계 | 제어 흐름 & 상태 | MCP · RAG 구현 | 평가 · 운영 전략 | MVP Project |
| Pain→Task→Tool<br>프롬프트 전략<br>MCP·RAG 판단 | **LangGraph**<br>**Tool Validation**<br>**리팩토링** | MCP 고급 설계<br>RAG 튜닝<br>Hybrid | 품질 평가<br>모니터링<br>서비스 아키텍처 | MVP 설계<br>구현·시연<br>발표 |

# Day 2 시간표

| 시간 | 내용 | 실습 |
| --- | --- | --- |
| 08:00–08:20 | Langfuse 셀프 호스팅 | Docker Compose |
| 08:20–09:30 | 프레임워크 기초 (I DO) | `00_basics/` 00~04 |
| 09:30–10:00 | Deep Agents + 비교 | `05~06` 노트북 |
| 10:00–10:10 | 휴식 | |
| 10:10–12:00 | Agent 4요소 · LangGraph 이론 | 가이드 Session 1–2 |
| 12:00–13:00 | 점심 | |
| 13:00–14:30 | LangGraph 워크플로 실습 | `01_langgraph_workflows/` |
| 14:30–14:40 | 휴식 | |

# 환경 세팅

```
# 1. 의존성 설치 (day2/ 디렉토리에서)
cd labs/day2
uv sync

# 2. 환경변수 설정
cp .env.example .env
# .env에 OPENAI_API_KEY=sk-... 입력

# 3. VS Code에서 노트북 열기
code .
```

VS Code에서 .ipynb 파일을 열고 Jupyter 커널을 선택하여 실행

# Langfuse 셀프 호스팅

LLM Observability를 사내망에서

Docker Compose로 20분 만에 구축

# 왜 Docker를 쓰는가

**직접 설치한다면**

**Docker를 쓰면**

# 왜 Docker를 쓰는가

**직접 설치한다면**

- PostgreSQL 설치

**Docker를 쓰면**

# 왜 Docker를 쓰는가

**직접 설치한다면**

- PostgreSQL 설치
- Redis 설치

**Docker를 쓰면**

# 왜 Docker를 쓰는가

**직접 설치한다면**

- PostgreSQL 설치
- Redis 설치
- ClickHouse 설치

**Docker를 쓰면**

# 왜 Docker를 쓰는가

**직접 설치한다면**

- PostgreSQL 설치
- Redis 설치
- ClickHouse 설치
- MinIO 설치

**Docker를 쓰면**

# 왜 Docker를 쓰는가

**직접 설치한다면**

- PostgreSQL 설치
- Redis 설치
- ClickHouse 설치
- MinIO 설치
- 버전 충돌, 포트 충돌…

**Docker를 쓰면**

# 왜 Docker를 쓰는가

**직접 설치한다면**

- PostgreSQL 설치
- Redis 설치
- ClickHouse 설치
- MinIO 설치
- 버전 충돌, 포트 충돌…

**Docker를 쓰면**

- `docker compose up -d`

# 왜 Docker를 쓰는가

### 직접 설치한다면

- PostgreSQL 설치
- Redis 설치
- ClickHouse 설치
- MinIO 설치
- 버전 충돌, 포트 충돌…

### Docker를 쓰면

- `docker compose up -d`
- 끝

# 왜 Docker를 쓰는가

**직접 설치한다면**

- PostgreSQL 설치
- Redis 설치
- ClickHouse 설치
- MinIO 설치
- 버전 충돌, 포트 충돌…

**Docker를 쓰면**

- `docker compose up -d`
- 끝

각 서비스를 **컨테이너**라는 격리된 상자에 넣어 실행 — 내 PC 환경 오염 없음

# Docker Desktop은 왜 설치하는가

# Docker Desktop은 왜 설치하는가

- Docker는 원래 **Linux 전용** 기술

# Docker Desktop은 왜 설치하는가

- Docker는 원래 **Linux 전용** 기술
- Windows/macOS에서 쓰려면 **가상 Linux 환경**이 필요

# Docker Desktop은 왜 설치하는가

- Docker는 원래 **Linux 전용** 기술
- Windows/macOS에서 쓰려면 **가상 Linux 환경**이 필요
- Docker Desktop이 이 환경을 **자동으로 생성**

# Docker Desktop은 왜 설치하는가

- Docker는 원래 **Linux 전용** 기술
- Windows/macOS에서 쓰려면 **가상 Linux 환경**이 필요
- Docker Desktop이 이 환경을 **자동으로 생성**
- GUI 대시보드로 컨테이너 상태를 시각적으로 확인

# Docker Desktop은 왜 설치하는가

- Docker는 원래 **Linux 전용** 기술
- Windows/macOS에서 쓰려면 **가상 Linux 환경**이 필요
- Docker Desktop이 이 환경을 **자동으로 생성**
- GUI 대시보드로 컨테이너 상태를 시각적으로 확인

Docker Desktop이 실행 중이어야 **docker 명령이 동작**한다

# Docker Compose 핵심 명령

| 명령 | 역할 |
| --- | --- |
| `docker compose up -d` | 모든 서비스 시작 (백그라운드) |
| `docker compose ps` | 서비스 상태 확인 |
| `docker compose logs -f` | 실시간 로그 확인 |
| `docker compose down` | 모든 서비스 중지 |
| `docker compose down -v` | 중지 + **데이터 삭제** |

# Docker Compose 핵심 명령

| 명령 | 역할 |
| --- | --- |
| `docker compose up -d` | 모든 서비스 시작 (백그라운드) |
| `docker compose ps` | 서비스 상태 확인 |
| `docker compose logs -f` | 실시간 로그 확인 |
| `docker compose down` | 모든 서비스 중지 |
| `docker compose down -v` | 중지 + **데이터 삭제** |

**compose.yml** = 어떤 서비스를 어떻게 실행할지 적은 레시피

# Day 1에서 LangSmith를 배운 이유

# Day 1에서 LangSmith를 배운 이유

- LLM Observability 플랫폼은 **대부분 비슷한 기능**을 제공

# Day 1에서 LangSmith를 배운 이유

- LLM Observability 플랫폼은 **대부분 비슷한 기능**을 제공
- LangSmith로 개념을 익히면 **다른 도구에도 그대로 적용**

# Day 1에서 LangSmith를 배운 이유

- LLM Observability 플랫폼은 **대부분 비슷한 기능**을 제공
- LangSmith로 개념을 익히면 **다른 도구에도 그대로 적용**
- 트레이싱, 프롬프트 관리, 평가 — 핵심 개념은 동일

# Day 1에서 LangSmith를 배운 이유

- LLM Observability 플랫폼은 **대부분 비슷한 기능**을 제공
- LangSmith로 개념을 익히면 **다른 도구에도 그대로 적용**
- 트레이싱, 프롬프트 관리, 평가 — 핵심 개념은 동일

"그런데 클라우드 서비스라 사내에서 못 쓰는 거 아닌가?"
→ 그래서 오늘은 **직접 설치하는 Langfuse**로 전환합니다

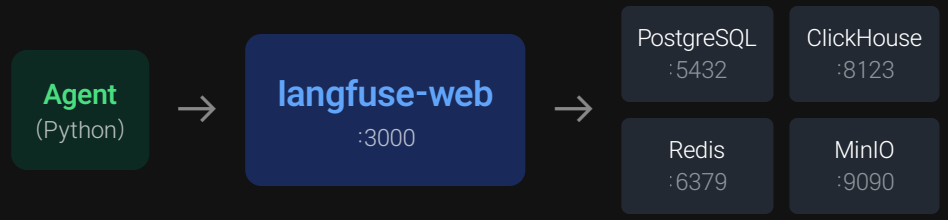
# LangSmith vs Langfuse — 역할 비교

| 기준 | LangSmith | Langfuse |
|---|---|---|
| **생태계 통합** | LangChain 네이티브 | CallbackHandler |
| **배포 환경** | 클라우드 전용 | 클라우드 + 셀프 호스팅 |
| **평가** | Online + Offline | Online + Offline |
| **프롬프트 관리** | Hub + Playground | 버전 관리 + Playground |
| **Agent 배포** | Fleet / Studio | 미지원 |
| **프레임워크 지원** | 다중 (OpenAI, Anthropic 등) | 50+ (OpenTelemetry) |
| **라이선스** | 상용 (무료 티어) | 오픈소스 (MIT) |

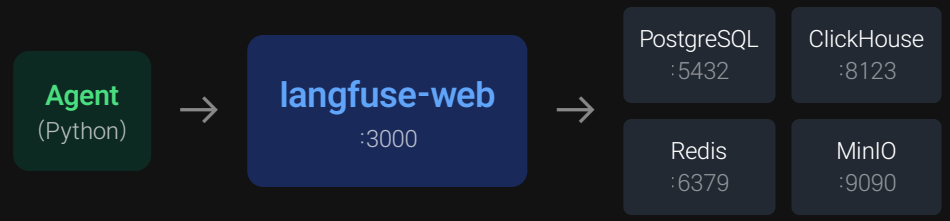
# LangSmith vs Langfuse — 역할 비교

| 기준 | LangSmith | Langfuse |
|---|---|---|
| **생태계 통합** | LangChain 네이티브 | CallbackHandler |
| **배포 환경** | 클라우드 전용 | 클라우드 + 셀프 호스팅 |
| **평가** | Online + Offline | Online + Offline |
| **프롬프트 관리** | Hub + Playground | 버전 관리 + Playground |
| **Agent 배포** | Fleet / Studio | 미지원 |
| **프레임워크 지원** | 다중 (OpenAI, Anthropic 등) | 50+ (OpenTelemetry) |
| **라이선스** | 상용 (무료 티어) | 오픈소스 (MIT) |

# Langfuse 아키텍처



# Langfuse 아키텍처




**compose.yml** 하나로 5개 서비스가 자동 실행

# Step 1 — Docker Compose 실행

```
# Docker Desktop이 실행 중인지 확인
docker --version

# langfuse-self-host 디렉토리로 이동
cd labs/day2/langfuse-self-host

# 백그라운드 실행 (첫 실행: 이미지 다운로드 3~5분)
docker compose up -d
```

# Step 1 — Docker Compose 실행

```
# Docker Desktop이 실행 중인지 확인
docker --version

# langfuse-self-host 디렉토리로 이동
cd labs/day2/langfuse-self-host

# 백그라운드 실행 (첫 실행: 이미지 다운로드 3~5분)
docker compose up -d
```

```
# 진행 상황 확인
docker compose logs -f langfuse-web

# 모든 서비스 상태 확인
docker compose ps
```

# Step 2 — 계정 생성 & API 키

# Step 2 — 계정 생성 & API 키

- 브라우저에서 `http://localhost:3000` 접속

# Step 2 — 계정 생성 & API 키

- 브라우저에서 `http://localhost:3000` 접속
- **Sign Up** → 이메일/비밀번호 입력 (로컬이라 아무 값 OK)

# Step 2 — 계정 생성 & API 키

- 브라우저에서 `http://localhost:3000` 접속
- **Sign Up** → 이메일/비밀번호 입력 (로컬이라 아무 값 OK)
- Settings → API Keys → **Create API Key**

# Step 2 — 계정 생성 & API 키

- 브라우저에서 `http://localhost:3000` 접속
- **Sign Up** → 이메일/비밀번호 입력 (로컬이라 아무 값 OK)
- Settings → API Keys → **Create API Key**
- `Secret Key`, `Public Key` 복사

# Step 2 — 계정 생성 & API 키

- 브라우저에서 `http://localhost:3000` 접속
- **Sign Up** → 이메일/비밀번호 입력 (로컬이라 아무 값 OK)
- Settings → API Keys → **Create API Key**
- `Secret Key`, `Public Key` 복사

```
# labs/day2/.env에 추가
LANGFUSE_SECRET_KEY=sk-lf- ...
LANGFUSE_PUBLIC_KEY=pk-lf- ...
LANGFUSE_HOST=http://localhost:3000
```

# 연동 확인

```
# 00_setup.ipynb 실행 시 아래가 출력되면 성공
# "Langfuse tracing ON — http://localhost:3000"
```

# 연동 확인

```
# 00_setup.ipynb 실행 시 아래가 출력되면 성공
# "Langfuse tracing ON — http://localhost:3000"
```

# 연동 확인

```
# 00_setup.ipynb 실행 시 아래가 출력되면 성공
# "Langfuse tracing ON — http://localhost:3000"
```

# 연동 확인

```
# 00_setup.ipynb 실행 시 아래가 출력되면 성공
# "Langfuse tracing ON — http://localhost:3000"
```

# 연동 확인

```
# 00_setup.ipynb 실행 시 아래가 출력되면 성공
# "Langfuse tracing ON — http://localhost:3000"
```

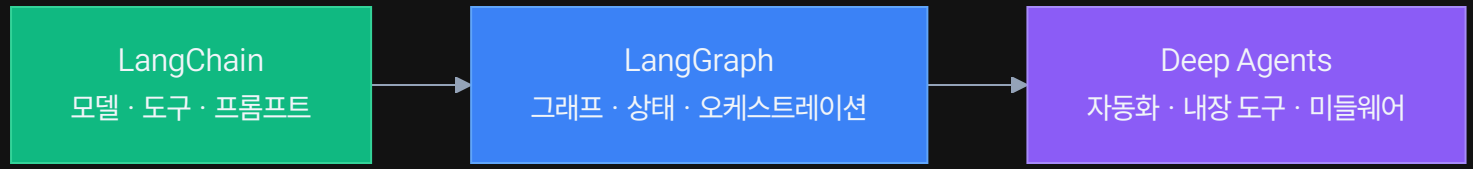
- 이후 모든 `agent.invoke()` 호출이 **자동으로 Trace**됨
- `localhost:3000` 에서 실시간 확인 가능
- 실습 종료 후: `docker compose down`

# 프레임워크 기초

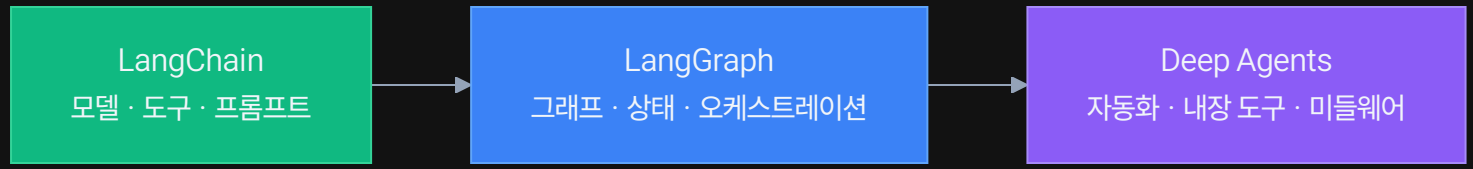
LangChain · LangGraph · Deep Agents

실습: 00 _ basics 노트북

# 3개 프레임워크의 계층 구조

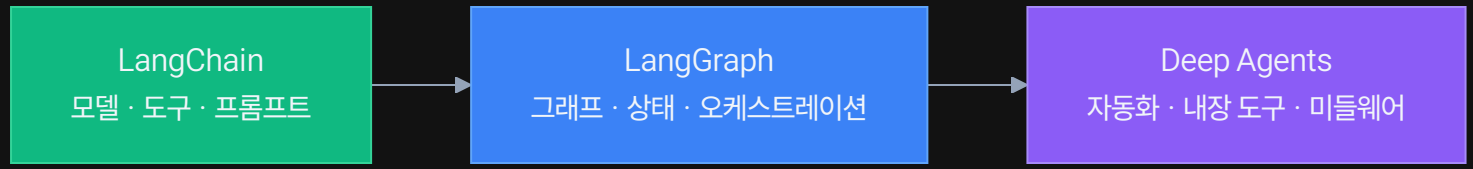


# 3개 프레임워크의 계층 구조



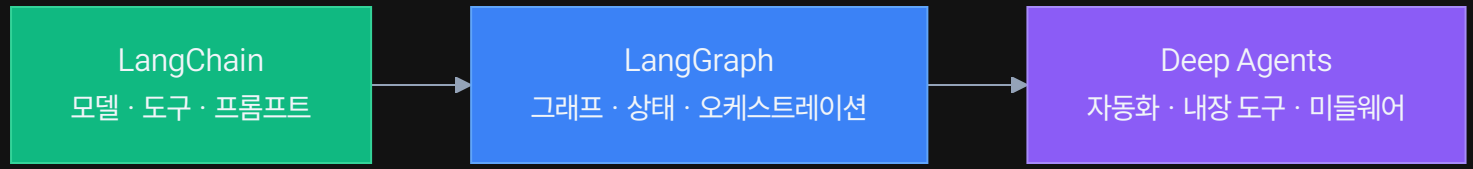

- **LangChain** = 부품 (엔진, 바퀴)

# 3개 프레임워크의 계층 구조




- **LangChain** = 부품 (엔진, 바퀴)
- **LangGraph** = 설계도 (회로 다이어그램)

# 3개 프레임워크의 계층 구조




- **LangChain** = 부품 (엔진, 바퀴)
- **LangGraph** = 설계도 (회로 다이어그램)
- **Deep Agents** = 완성차 (운전만 하면 됨)

# LangChain — 기반 레이어

```python
from langchain.tools import tool
from langchain.chat_models import init_chat_model
from langgraph.prebuilt import create_react_agent

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

model = init_chat_model("gpt-5.4")
agent = create_react_agent(model, tools=[multiply])
```

**@tool** 데코레이터 하나로 LLM이 호출 가능한 도구가 된다

# LangChain — 기반 레이어

```python
from langchain.tools import tool
from langchain.chat_models import init_chat_model
from langgraph.prebuilt import create_react_agent

@tool
def multiply(a: int, b: int) → int:
    """두 수를 곱합니다."""
    return a * b

model = init_chat_model("gpt-5.4")
agent = create_react_agent(model, tools=[multiply])
```

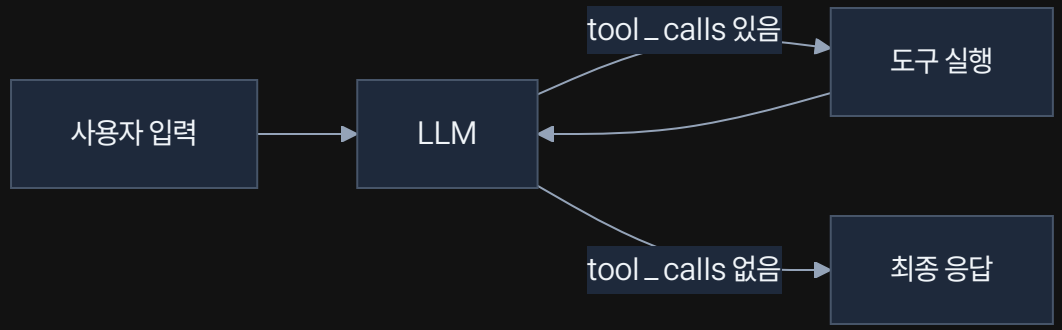
**@tool** 데코레이터 하나로 LLM이 호출 가능한 도구가 된다

# LangChain — 기반 레이어

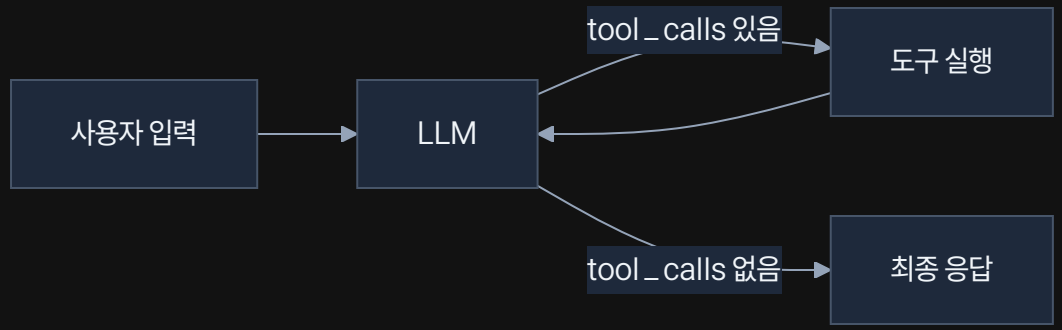
```python
from langchain.tools import tool
from langchain.chat_models import init_chat_model
from langgraph.prebuilt import create_react_agent

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

model = init_chat_model("gpt-5.4")
agent = create_react_agent(model, tools=[multiply])
```

**@tool** 데코레이터 하나로 LLM이 호출 가능한 도구가 된다

# LangChain — ReAct 자동 루프

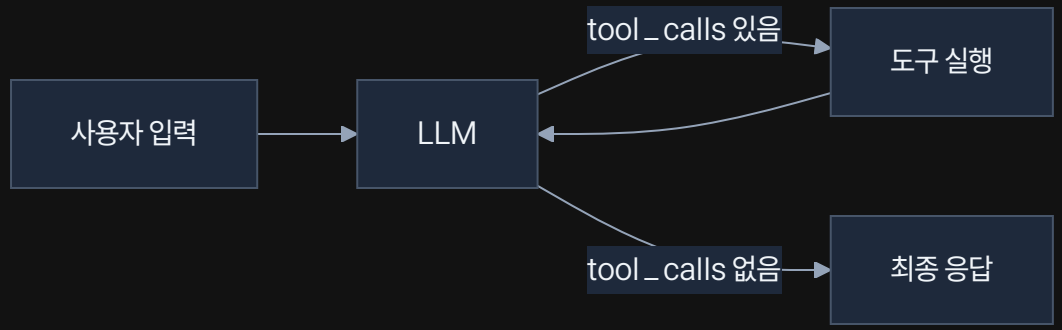


# LangChain — ReAct 자동 루프



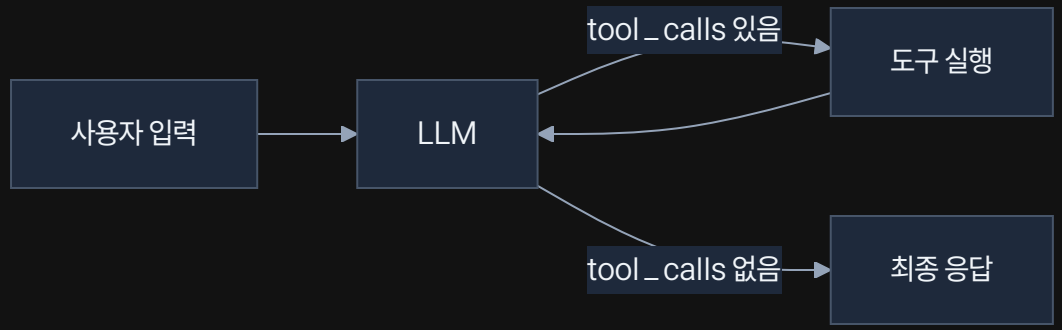

- LLM이 도구 호출 여부를 **스스로 판단**

# LangChain — ReAct 자동 루프




- LLM이 도구 호출 여부를 **스스로 판단**
- 도구 결과를 받아 다시 LLM에 전달

# LangChain — ReAct 자동 루프




- LLM이 도구 호출 여부를 **스스로 판단**
- 도구 결과를 받아 다시 LLM에 전달
- tool_calls가 없으면 자동 종료

# LangGraph — 오케스트레이션

```python
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator
from langchain_core.messages import AnyMessage

class State(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]

def llm_node(state: State) -> dict:
    result = model.invoke(state["messages"])
    return {"messages": [result]}

graph = StateGraph(State)
graph.add_node("llm", llm_node)
```

모든 Node는 **State를 받아서 State를 반환**한다

# LangGraph — 오케스트레이션

```python
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator
from langchain_core.messages import AnyMessage

class State(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]

def llm_node(state: State) -> dict:
    result = model.invoke(state["messages"])
    return {"messages": [result]}

graph = StateGraph(State)
graph.add_node("llm", llm_node)
```

모든 Node는 **State를 받아서 State를 반환**한다

# LangGraph — 오케스트레이션

```python
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator
from langchain_core.messages import AnyMessage

class State(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]

def llm_node(state: State) -> dict:
    result = model.invoke(state["messages"])
    return {"messages": [result]}

graph = StateGraph(State)
graph.add_node("llm", llm_node)
```

모든 Node는 **State를 받아서 State를 반환**한다

# Deep Agents — 완성형 하네스

```python
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

agent = create_deep_agent(
    model=ChatOpenAI(model="gpt-5.4"),
    system_prompt="데이터 분석가입니다.",
)

# 내장 도구: ls, read_file, write_file, edit_file, glob, grep, write_todos, task
```

# Deep Agents — 완성형 하네스

```python
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

agent = create_deep_agent(
    model=ChatOpenAI(model="gpt-5.4"),
    system_prompt="데이터 분석가입니다.",
)

# 내장 도구: ls, read_file, write_file, edit_file, glob, grep, write_todos, task
```

# Deep Agents — 완성형 하네스

```python
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

agent = create_deep_agent(
    model=ChatOpenAI(model="gpt-5.4"),
    system_prompt="데이터 분석가입니다.",
)

# 내장 도구: ls, read_file, write_file, edit_file, glob, grep, write_todos, task
```

# Deep Agents — 완성형 하네스

```python
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

agent = create_deep_agent(
    model=ChatOpenAI(model="gpt-5.4"),
    system_prompt="데이터 분석가입니다.",
)

# 내장 도구: ls, read_file, write_file, edit_file, glob, grep, write_todos, task
```

- 파일 도구 6종 + 계획 도구 + 서브에이전트 **자동 포함**

# Deep Agents — 완성형 하네스

```python
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

agent = create_deep_agent(
    model=ChatOpenAI(model="gpt-5.4"),
    system_prompt="데이터 분석가입니다.",
)

# 내장 도구: ls, read_file, write_file, edit_file, glob, grep, write_todos, task
```

- 파일 도구 6종 + 계획 도구 + 서브에이전트 **자동 포함**
- `backend` 파라미터로 저장소 전환 (State → Filesystem → Store)

# Deep Agents — 완성형 하네스

```python
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

agent = create_deep_agent(
    model=ChatOpenAI(model="gpt-5.4"),
    system_prompt="데이터 분석가입니다.",
)

# 내장 도구: ls, read_file, write_file, edit_file, glob, grep, write_todos, task
```

- 파일 도구 6종 + 계획 도구 + 서브에이전트 **자동 포함**
- `backend` 파라미터로 저장소 전환 (State → Filesystem → Store)
- LangGraph의 `CompiledStateGraph` 를 반환

# 프레임워크 선택 기준

| 기준 | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| 복잡도 | 낮음 | 중간 | 높음 (자동화) |
| 제어 수준 | ReAct 자동 | 노드 단위 | 미들웨어 |
| 파일 I/O | 수동 | 수동 | **내장** |
| 멀티에이전트 | 핸드오프 | 서브그래프 | 서브에이전트 |
| 계획 수립 | 수동 | 수동 | **write_todos** |

# 프레임워크 선택 기준

| 기준 | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| 복잡도 | 낮음 | 중간 | 높음 (자동화) |
| 제어 수준 | ReAct 자동 | 노드 단위 | 미들웨어 |
| 파일 I/O | 수동 | 수동 | **내장** |
| 멀티에이전트 | 핸드오프 | 서브그래프 | 서브에이전트 |
| 계획 수립 | 수동 | 수동 | **write_todos** |

단순 Tool 호출 → **LangChain** · 복잡한 분기 → **LangGraph** · 파일 작업 → **Deep Agents**

# 실습: 프레임워크 기초 체험

**00_basics 노트북 순서**

# 실습: 프레임워크 기초 체험

**00_basics 노트북 순서**

- `00_setup.ipynb` — 환경 설정, Langfuse 연동 (15분)

# 실습: 프레임워크 기초 체험

**00_basics 노트북 순서**

- `00_setup.ipynb` — 환경 설정, Langfuse 연동 (15분)
- `01_llm_basics.ipynb` — 메시지 구조, 스트리밍 (15분)

# 실습: 프레임워크 기초 체험

**00_basics 노트북 순서**

- `00_setup.ipynb` — 환경 설정, Langfuse 연동 (15분)
- `01_llm_basics.ipynb` — 메시지 구조, 스트리밍 (15분)
- `02_langchain_basics.ipynb` — @tool, ReAct Agent (20분)

# 실습: 프레임워크 기초 체험

**00_basics 노트북 순서**

- `00_setup.ipynb` — 환경 설정, Langfuse 연동 (15분)
- `01_llm_basics.ipynb` — 메시지 구조, 스트리밍 (15분)
- `02_langchain_basics.ipynb` — @tool, ReAct Agent (20분)
- `03_langchain_memory.ipynb` — InMemorySaver, thread_id (15분)

# 실습: 프레임워크 기초 체험

**00_basics 노트북 순서**

- `00_setup.ipynb` — 환경 설정, Langfuse 연동 (15분)
- `01_llm_basics.ipynb` — 메시지 구조, 스트리밍 (15분)
- `02_langchain_basics.ipynb` — @tool, ReAct Agent (20분)
- `03_langchain_memory.ipynb` — InMemorySaver, thread_id (15분)
- `04_langgraph_basics.ipynb` — StateGraph, Node-Edge (25분)

# 실습: Deep Agents + 비교

# 실습: Deep Agents + 비교

- `05_deep_agents_basics.ipynb` — create_deep_agent, 내장 도구, Backend (20분)

# 실습: Deep Agents + 비교

- `05_deep_agents_basics.ipynb` — create_deep_agent, 내장 도구, Backend (20분)
- `06_comparison.ipynb` — 3 프레임워크 동일 작업 비교 (10분)

# 실습: Deep Agents + 비교

- `05_deep_agents_basics.ipynb` — create_deep_agent, 내장 도구, Backend (20분)
- `06_comparison.ipynb` — 3 프레임워크 동일 작업 비교 (10분)

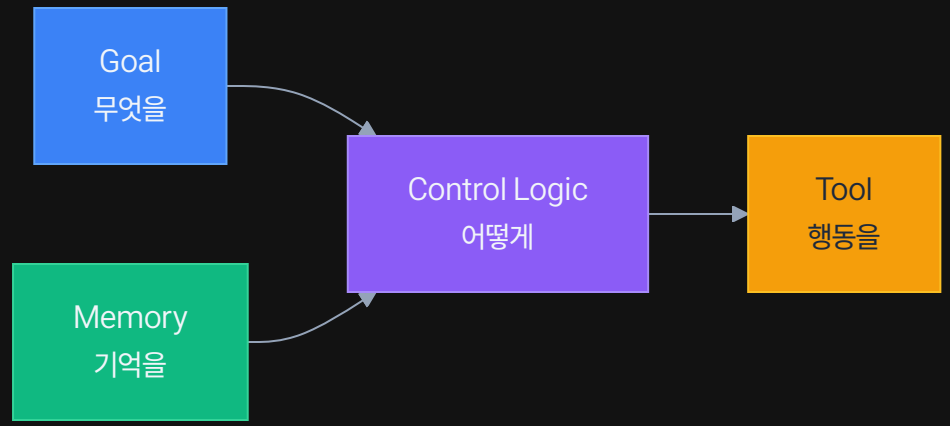
같은 "웹 검색 Agent"가 프레임워크마다 **어떻게 달라지는지** 직접 확인

# Agent 4요소

Goal · Memory · Tool · Control Logic

Session 1 이론

# Agent = 4가지 요소의 조합




| 요소 | 핵심 질문 |
|---|---|
| **Goal** | "성공 조건은 무엇인가?" |
| **Memory** | "어떤 컨텍스트를 유지하는가?" |
| **Tool** | "어떤 외부 능력이 필요한가?" |

# Goal — 목표 정의

```
# 나쁜 예: 목표가 모호함
goal = "데이터를 분석해"

# 좋은 예: 조건이 명확함
goal = Goal(
    description="판매 데이터에서 이상치를 탐지한다",
    success_condition="이상치 목록 + 근거 보고서 생성 완료",
    abort_condition="데이터 접근 실패 또는 3회 재시도 초과",
)
```

# Goal — 목표 정의

```
# 나쁜 예: 목표가 모호함
goal = "데이터를 분석해"

# 좋은 예: 조건이 명확함
goal = Goal(
    description="판매 데이터에서 이상치를 탐지한다",
    success_condition="이상치 목록 + 근거 보고서 생성 완료",
    abort_condition="데이터 접근 실패 또는 3회 재시도 초과",
)
```

# Goal — 목표 정의

```
# 나쁜 예: 목표가 모호함
goal = "데이터를 분석해"

# 좋은 예: 조건이 명확함
goal = Goal(
    description="판매 데이터에서 이상치를 탐지한다",
    success_condition="이상치 목록 + 근거 보고서 생성 완료",
    abort_condition="데이터 접근 실패 또는 3회 재시도 초과",
)
```

잘못 정의된 Goal은 **무한 루프**의 주요 원인

# Goal — 프레임워크별 구현

| | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| 방식 | SystemMessage | SystemMessage | system_prompt |
| 확장 | — | — | AGENTS.md |

# Goal — 프레임워크별 구현

| | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| 방식 | SystemMessage | SystemMessage | system_prompt |
| 확장 | — | — | AGENTS.md |

```
# Deep Agents — 문서로 Goal 관리 (코드 수정 없이 변경 가능)
agent = create_deep_agent(
    model=model,
    system_prompt="판매 이상치를 탐지합니다.",
    memory=["/project/AGENTS.md"],  # 영구 규칙 자동 주입
)
```

# Memory — 4가지 레이어

```
┌──────────────────────────────────┐
│ Episodic Memory   (이번 대화/세션)   │ ← 가장 빠름, 가장 휘발성
├──────────────────────────────────┤
│ Working Memory    (현재 Task 상태)  │ ← Agent State에 보관
├──────────────────────────────────┤
│ Semantic Memory   (도메인 지식)      │ ← Vector DB, RAG
├──────────────────────────────────┤
│ Procedural Memory (실행 패턴)        │ ← 프롬프트, Few-shot
└──────────────────────────────────┘
```

# Memory — 4가지 레이어

```
┌──────────────────────────────────┐
│ Episodic Memory   (이번 대화/세션)   │ ← 가장 빠름, 가장 휘발성
├──────────────────────────────────┤
│ Working Memory    (현재 Task 상태)  │ ← Agent State에 보관
├──────────────────────────────────┤
│ Semantic Memory   (도메인 지식)      │ ← Vector DB, RAG
├──────────────────────────────────┤
│ Procedural Memory (실행 패턴)        │ ← 프롬프트, Few-shot
└──────────────────────────────────┘
```

**Working Memory**에 무엇을 담을지 결정하는 것이 설계의 핵심

# Memory — 프레임워크별 구현

| | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| 단기 | InMemorySaver | 동일 | 동일 |
| 장기 | InMemoryStore (수동) | Store (수동) | CompositeBackend (자동) |

# Memory — 프레임워크별 구현

| | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| 단기 | InMemorySaver | 동일 | 동일 |
| 장기 | InMemoryStore (수동) | Store (수동) | CompositeBackend (자동) |

```
# Deep Agents — 경로 기반 자동 라우팅
backend=lambda rt: CompositeBackend(
    default=StateBackend(rt),                # / → 임시
    routes={"/memories/": StoreBackend(rt)}, # /memories/ → 영속
)
# /memories/에 쓰면 자동으로 장기 저장!
```

# Tool — 원자적 설계

## 나쁜 예

```python
# 하나의 Tool이 3가지 역할
def search_and_summarize(q):
    results = web_search(q)
    summary = llm_summarize(results)
    save_to_db(summary)
    return summary
```

## 좋은 예

```python
# 각 Tool은 하나의 역할
@tool
def web_search(q: str) -> list:
    ...
@tool
def summarize(text: str) -> str:
    ...
@tool
def save_result(k: str, v: str):
    ...
```

# Tool — 프레임워크별 비교

| | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| 커스텀 도구 | @tool | @tool | @tool |
| 파일 I/O | 수동 | 수동 | **내장 6종** |
| 계획 수립 | 수동 | 수동 | **write_todos** |
| 서브에이전트 | 수동 | 서브그래프 | **task** |
| 셸 실행 | 수동 | 수동 | **execute** |

# Tool — 프레임워크별 비교

| | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| 커스텀 도구 | @tool | @tool | @tool |
| 파일 I/O | 수동 | 수동 | **내장 6종** |
| 계획 수립 | 수동 | 수동 | **write_todos** |
| 서브에이전트 | 수동 | 서브그래프 | **task** |
| 셸 실행 | 수동 | 수동 | **execute** |

Deep Agents는 **create_deep_agent()** 한 번이면 코딩 에이전트 완성

# Control Logic — 세 가지 패턴

# Control Logic — 세 가지 패턴

**LangChain**

ReAct 자동 루프

블랙박스 — LLM이 알아서 판단

# Control Logic — 세 가지 패턴

**LangChain**

ReAct 자동 루프

블랙박스 — LLM이 알아서 판단

**LangGraph**

Graph 명시적 제어

노드/엣지 단위로 흐름 정의

# Control Logic — 세 가지 패턴

### LangChain

ReAct 자동 루프

블랙박스 — LLM이 알아서 판단

### LangGraph

Graph 명시적 제어

노드/엣지 단위로 흐름 정의

### Deep Agents

Middleware 파이프라인

미들웨어로 인터셉트 + 확장

# Control Logic — 세 가지 패턴

**LangChain**

ReAct 자동 루프

블랙박스 — LLM이 알아서 판단

**LangGraph**

Graph 명시적 제어

노드/엣지 단위로 흐름 정의

**Deep Agents**

Middleware 파이프라인

미들웨어로 인터셉트 + 확장

제어 수준: **자동(LangChain) → 명시적(LangGraph) → 자동+확장(Deep Agents)**

# 4요소 종합 비교

| 요소 | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| Goal | SystemMessage | SystemMessage | system_prompt + AGENTS.md |
| Memory | 체크포인터 + 수동 | 체크포인터 + Store | CompositeBackend 자동 |
| Tool | @tool 직접 정의 | @tool + bind_tools | @tool + **내장 8종** |
| Control | ReAct 자동 | Graph 명시적 | Middleware 자동+확장 |

# 4요소 종합 비교

| 요소 | LangChain | LangGraph | Deep Agents |
|---|---|---|---|
| Goal | SystemMessage | SystemMessage | system_prompt + AGENTS.md |
| Memory | 체크포인터 + 수동 | 체크포인터 + Store | CompositeBackend 자동 |
| Tool | @tool 직접 정의 | @tool + bind_tools | @tool + **내장 8종** |
| Control | ReAct 자동 | Graph 명시적 | Middleware 자동+확장 |

세 프레임워크는 경쟁이 아니라 **계층 구조**다

# Single-step vs Multi-step

**Single-step**

**Multi-step**

# Single-step vs Multi-step

## Single-step

- LLM 호출 1회

## Multi-step

# Single-step vs Multi-step

## Single-step

- LLM 호출 **1회**
- 상태 관리 불필요

## Multi-step

# Single-step vs Multi-step

## Single-step

- LLM 호출 **1회**
- 상태 관리 불필요
- 분류, 요약, 번역

## Multi-step

# Single-step vs Multi-step

## Single-step

- LLM 호출 **1회**
- 상태 관리 불필요
- 분류, 요약, 번역
- 비용 낮음

## Multi-step

# Single-step vs Multi-step

### Single-step

- LLM 호출 **1회**
- 상태 관리 불필요
- 분류, 요약, 번역
- 비용 낮음

### Multi-step

- LLM 호출 **여러 번**

# Single-step vs Multi-step

### Single-step

- LLM 호출 **1회**
- 상태 관리 불필요
- 분류, 요약, 번역
- 비용 낮음

### Multi-step

- LLM 호출 **여러 번**
- 상태 관리 필수

# Single-step vs Multi-step

### Single-step

- LLM 호출 **1회**
- 상태 관리 불필요
- 분류, 요약, 번역
- 비용 낮음

### Multi-step

- LLM 호출 **여러 번**
- 상태 관리 필수
- 조사, 계획, 자동화

# Single-step vs Multi-step

### Single-step

- LLM 호출 **1회**
- 상태 관리 불필요
- 분류, 요약, 번역
- 비용 낮음

### Multi-step

- LLM 호출 **여러 번**
- 상태 관리 필수
- 조사, 계획, 자동화
- 실패 복구 가능

# Single-step vs Multi-step

### Single-step

- LLM 호출 **1회**
- 상태 관리 불필요
- 분류, 요약, 번역
- 비용 낮음

### Multi-step

- LLM 호출 **여러 번**
- 상태 관리 필수
- 조사, 계획, 자동화
- 실패 복구 가능

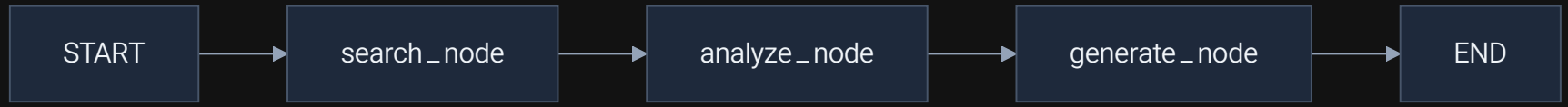
"LLM 한 번 호출로 끝나는가?" → Yes면 Single-step

# LangGraph 제어 흐름

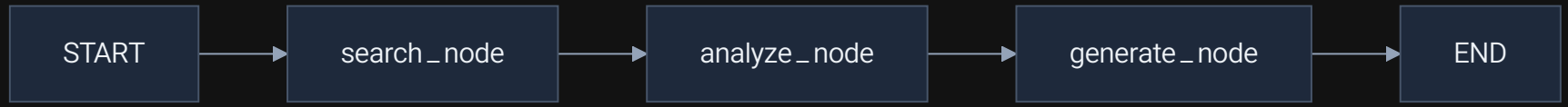
Node · Edge · State

Session 2 이론

# Node–Edge–State

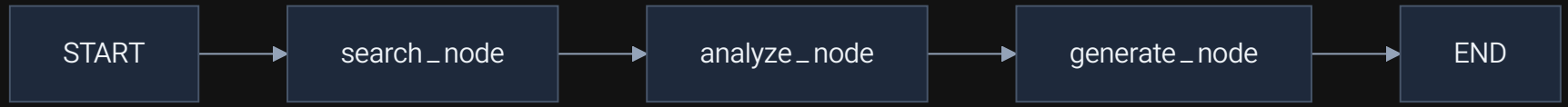


# Node–Edge–State



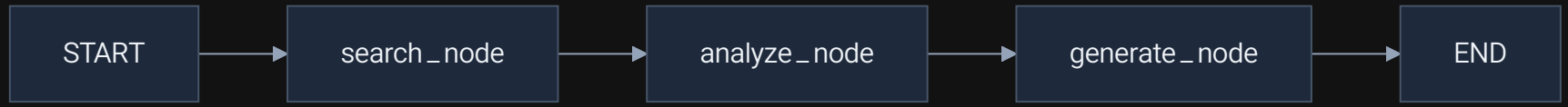

- **Node** = Python 함수 (State → dict)

# Node – Edge – State



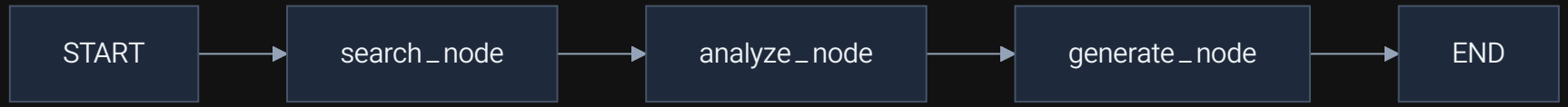

- **Node** = Python 함수 (State → dict)
- **Edge** = 다음 Node 연결

# Node – Edge – State




- **Node** = Python 함수 (State → dict)
- **Edge** = 다음 Node 연결
- **State** = 모든 Node가 공유하는 데이터

# Node – Edge – State




- **Node** = Python 함수 (State → dict)
- **Edge** = 다음 Node 연결
- **State** = 모든 Node가 공유하는 데이터

핵심 계약: Node는 **State를 받아서 dict를 반환**한다

# State 설계 — Annotated 리듀서

```python
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    query: str                                  # 단순 덮어쓰기
    messages: Annotated[list, operator.add]      # 리스트 누적
    retry_count: Annotated[int, operator.add]    # 카운터 누적
```

# State 설계 — Annotated 리듀서

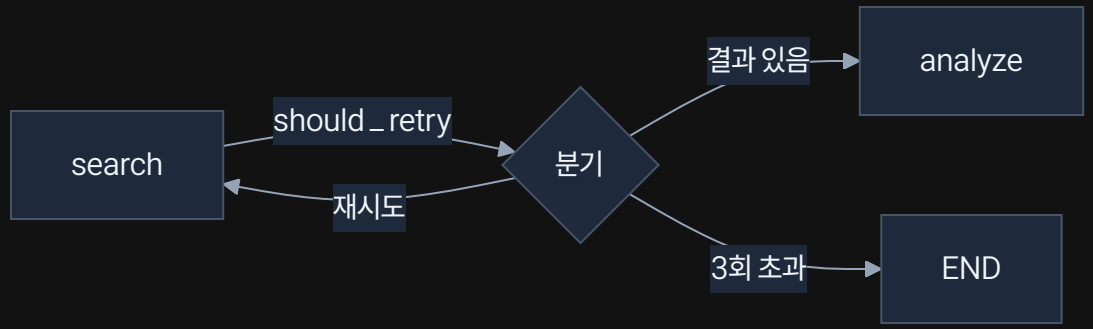
```python
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    query: str                                  # 단순 덮어쓰기
    messages: Annotated[list, operator.add]      # 리스트 누적
    retry_count: Annotated[int, operator.add]    # 카운터 누적
```

# State 설계 — Annotated 리듀서

```python
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    query: str                                  # 단순 덮어쓰기
    messages: Annotated[list, operator.add]      # 리스트 누적
    retry_count: Annotated[int, operator.add]    # 카운터 누적
```

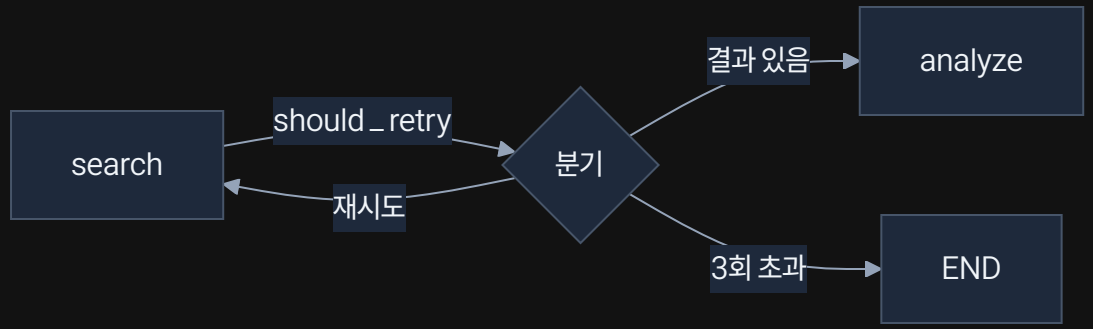
| 필드 | 현재 값 | 반환 값 | 결과 |
|---|---|---|---|
| query | "AI" | "ML" | "ML" (덮어쓰기) |
| messages | "a" | "b" | "a","b" (누적) |
| retry_count | 2 | 1 | 3 (합산) |

# 조건 분기 — Conditional Edge



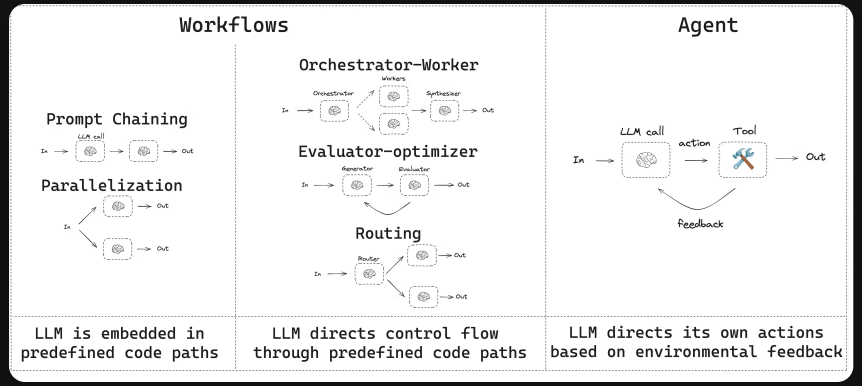

```python
def should_retry(state: WorkflowState) -> str:
    if state["retry_count"] ≥ 3:
        return "fail"
    if not state["search_results"]:
        return "retry"
    return "analyze"

graph.add_conditional_edges(
    "search",
    should_retry,
    {"analyze": "analyze", "retry": "search", "fail": END}
)
```

# 조건 분기 — Conditional Edge



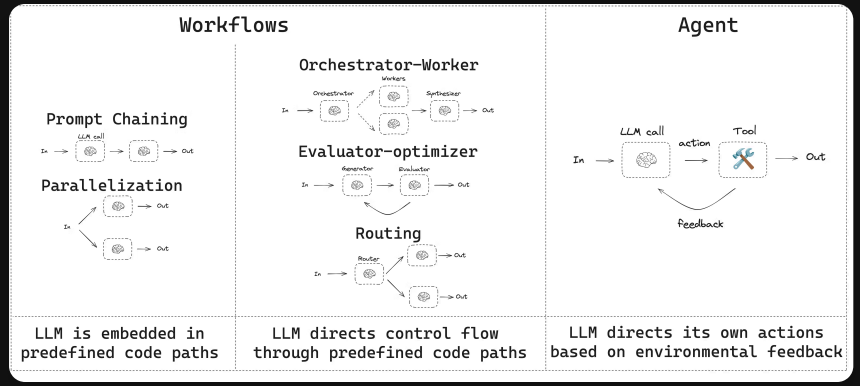

```python
def should_retry(state: WorkflowState) -> str:
    if state["retry_count"] ≥ 3:
        return "fail"
    if not state["search_results"]:
        return "retry"
    return "analyze"

graph.add_conditional_edges(
    "search",
    should_retry,
    {"analyze": "analyze", "retry": "search", "fail": END}
)
```

# Workflow vs Agent




출처: docs.langchain.com/oss/python/langgraph/workflows-agents

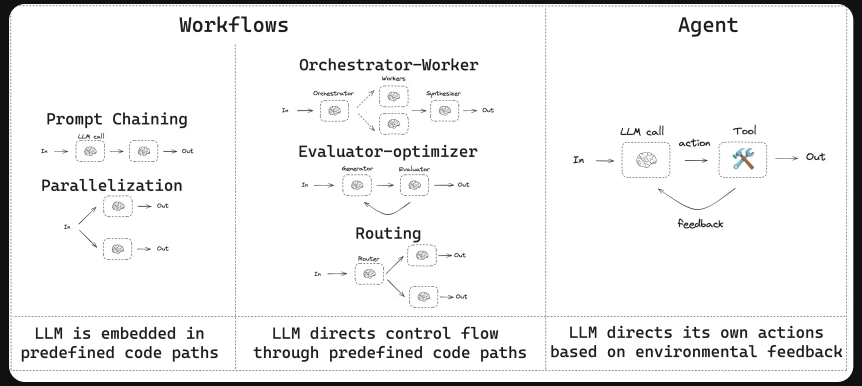
# Workflow vs Agent




- **Workflow** — 흐름이 코드로 고정

출처: docs.langchain.com/oss/python/langgraph/workflows-agents

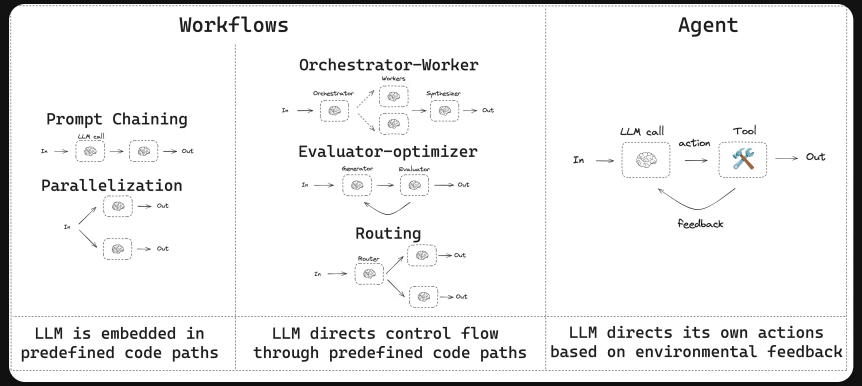
# Workflow vs Agent




- **Workflow** — 흐름이 코드로 고정
- **Agent** — LLM이 스스로 흐름 결정

출처: docs.langchain.com/oss/python/langgraph/workflows-agents

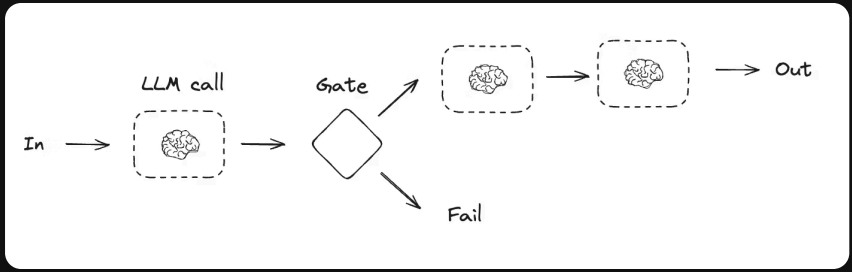
# Workflow vs Agent



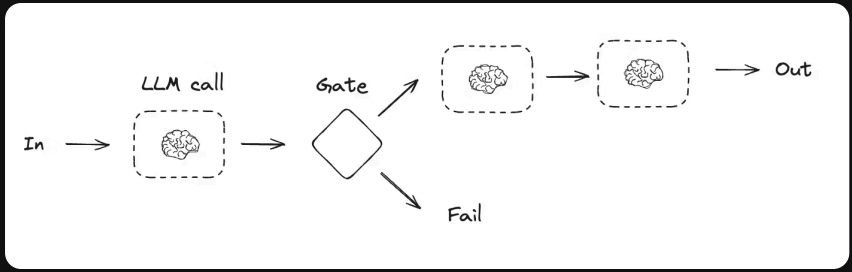

- **Workflow** — 흐름이 코드로 고정
- **Agent** — LLM이 스스로 흐름 결정
- 대부분의 실무는 **Workflow + Agent 혼합**

출처: docs.langchain.com/oss/python/langgraph/workflows-agents

# Prompt Chaining

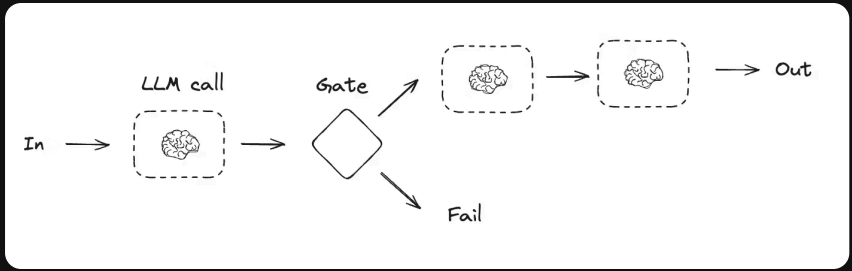


# Prompt Chaining



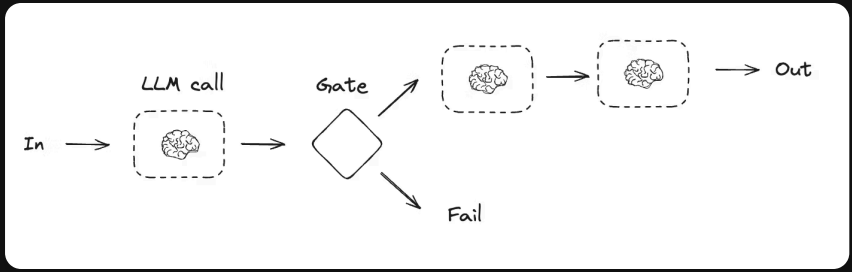

- LLM 호출을 **순차적으로 연결**

# Prompt Chaining



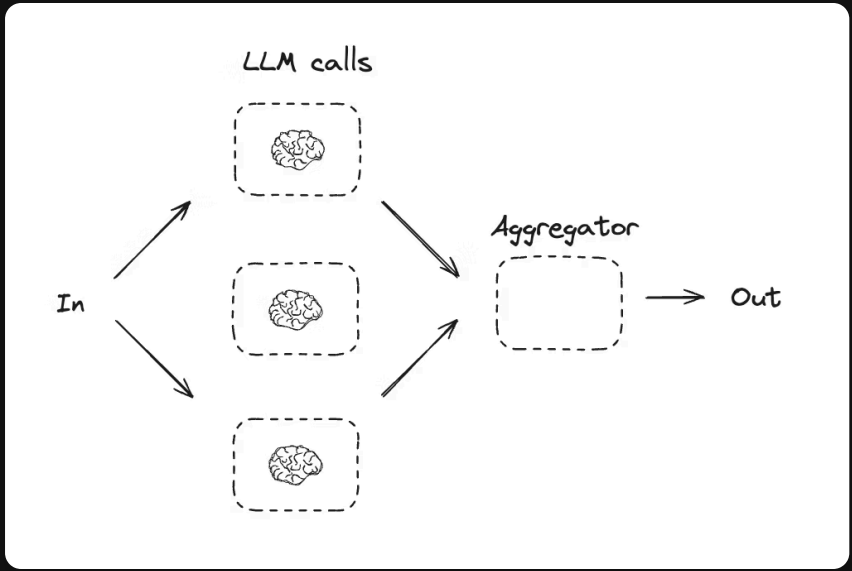

- LLM 호출을 **순차적으로 연결**
- 이전 출력이 다음 입력으로 전달

# Prompt Chaining

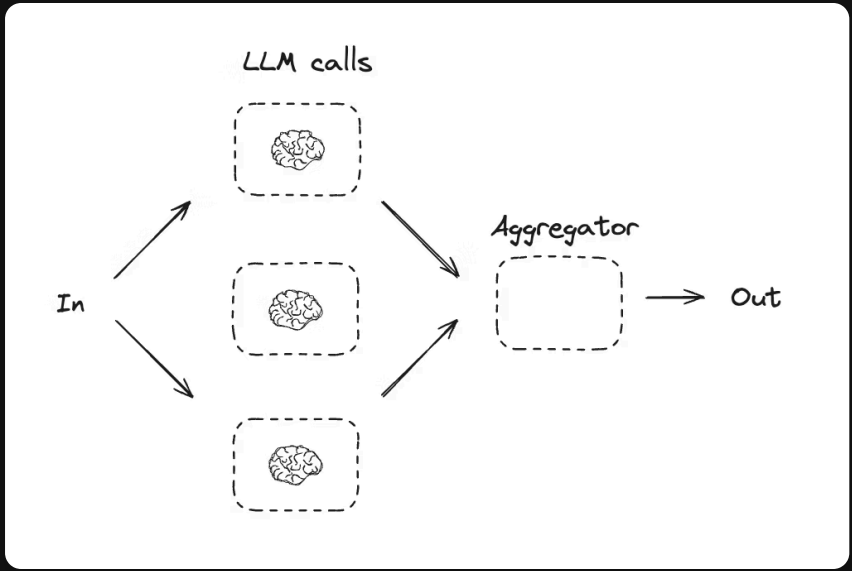



- LLM 호출을 **순차적으로 연결**
- 이전 출력이 다음 입력으로 전달
- 예: 번역 → 검수 → 포맷팅

# Parallelization

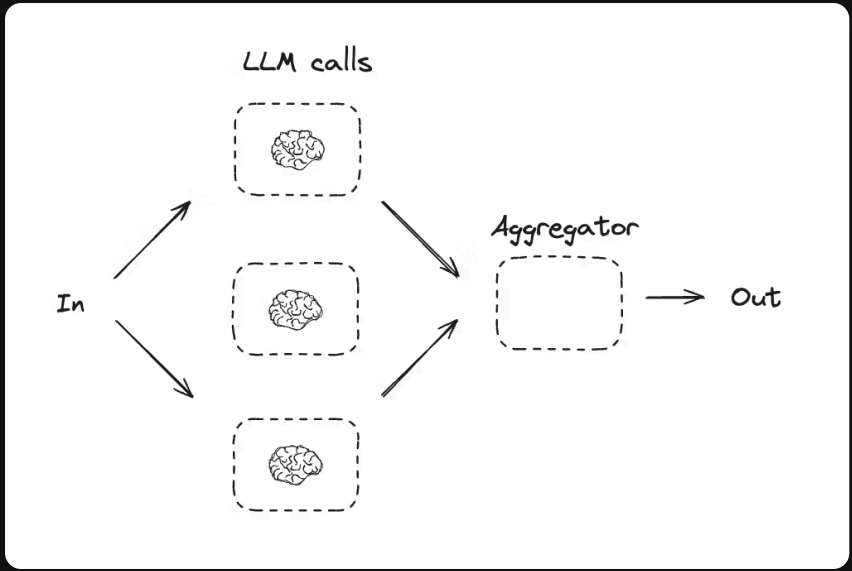


# Parallelization



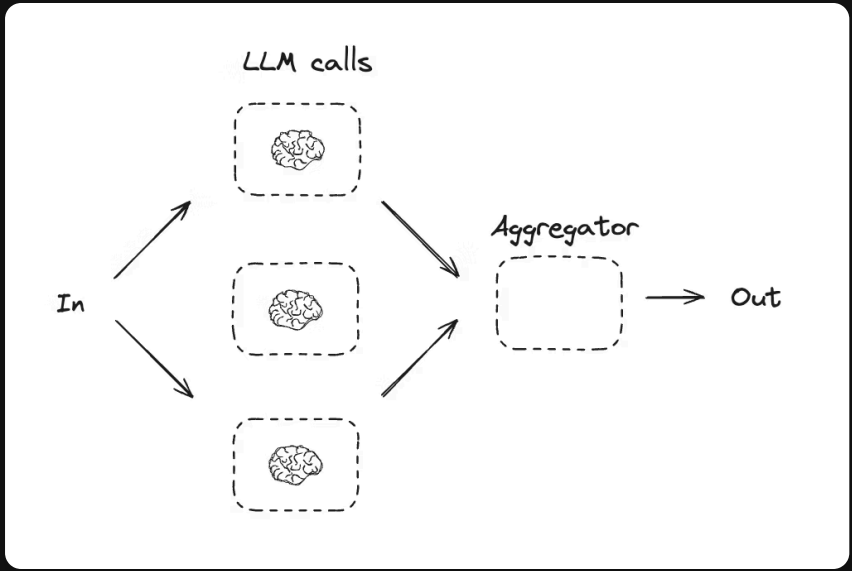

- 독립적인 작업을 **동시 실행**

# Parallelization



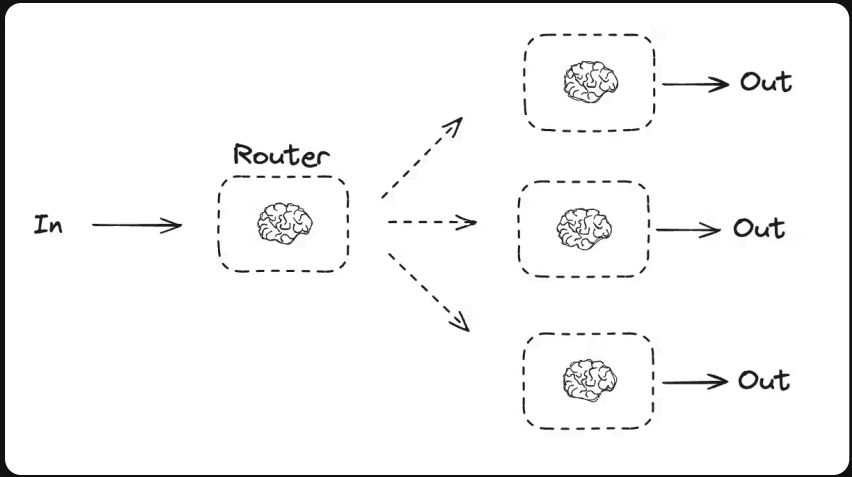

- 독립적인 작업을 **동시 실행**
- LangGraph의 `Send()` API 활용

# Parallelization

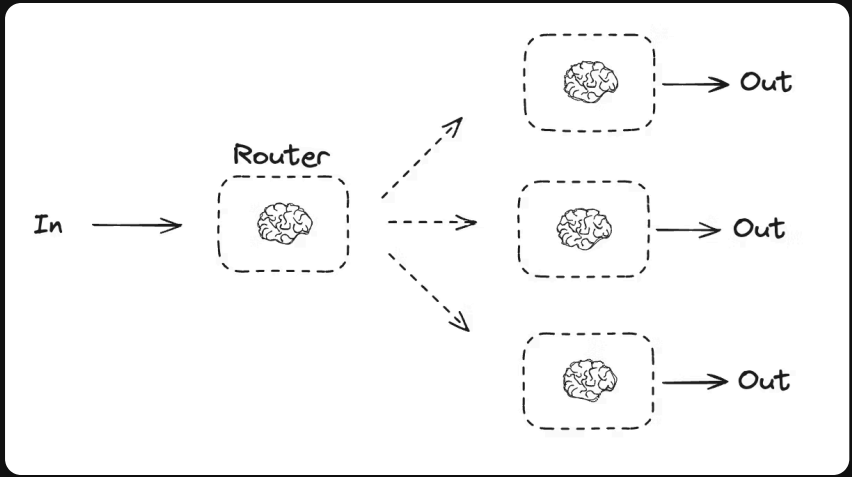



- 독립적인 작업을 **동시 실행**
- LangGraph의 `Send()` API 활용
- 예: 3개 URL 병렬 크롤링

# Routing

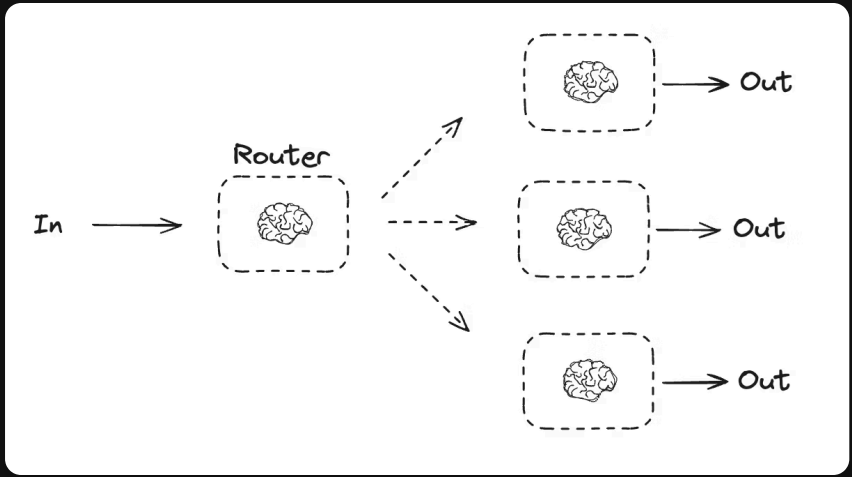


# Routing



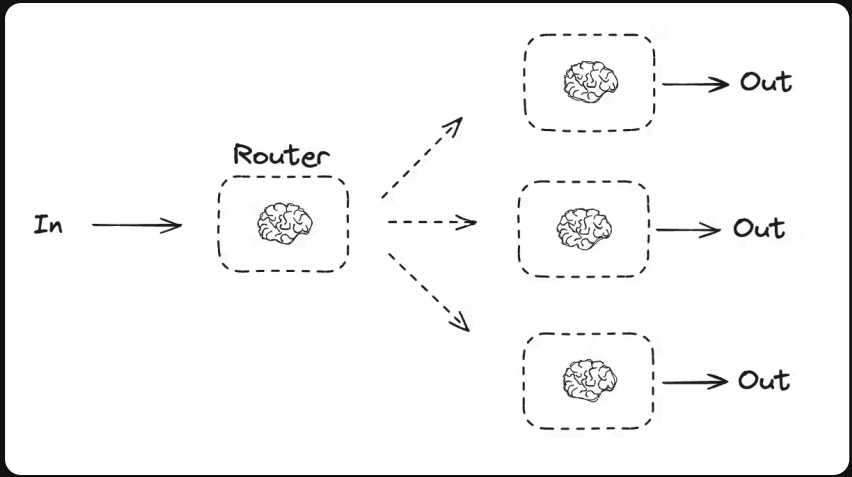

- 입력을 분류하여 **전문 경로로 분기**

# Routing



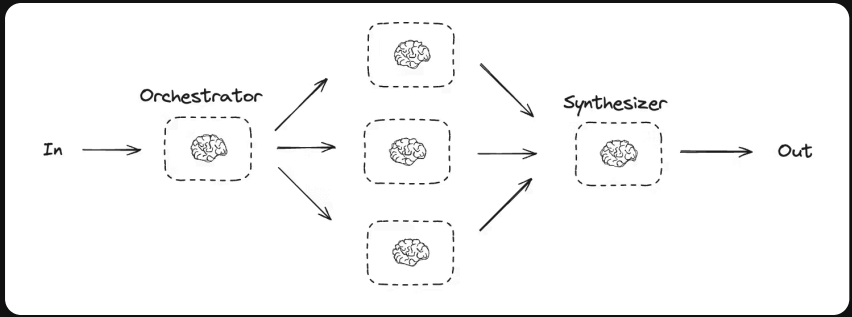

- 입력을 분류하여 **전문 경로로 분기**
- `add_conditional_edges` 활용

# Routing

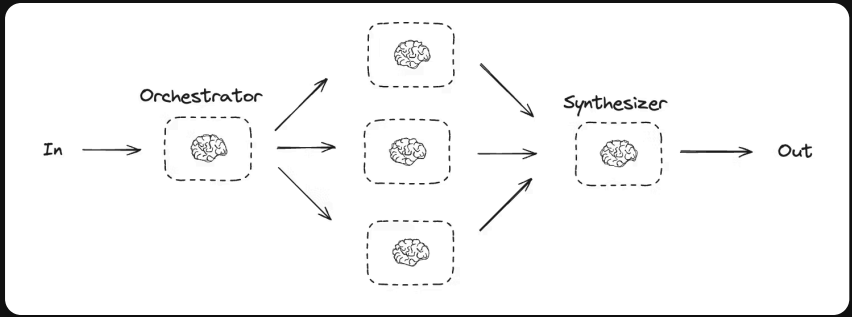



- 입력을 분류하여 **전문 경로로 분기**
- `add_conditional_edges` 활용
- 예: 쿼리 → 단순/코드/복잡 처리

# Orchestrator-Worker

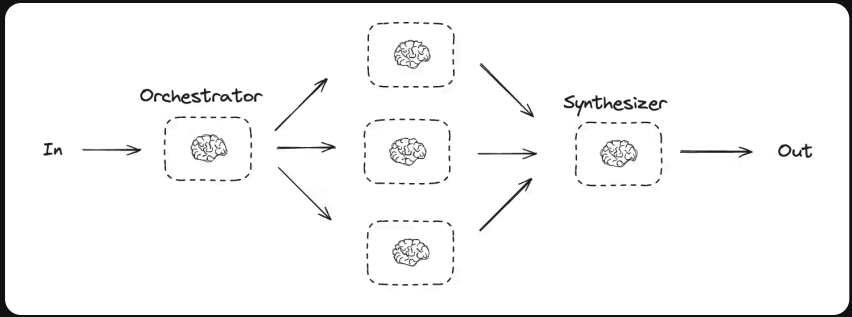


# Orchestrator-Worker



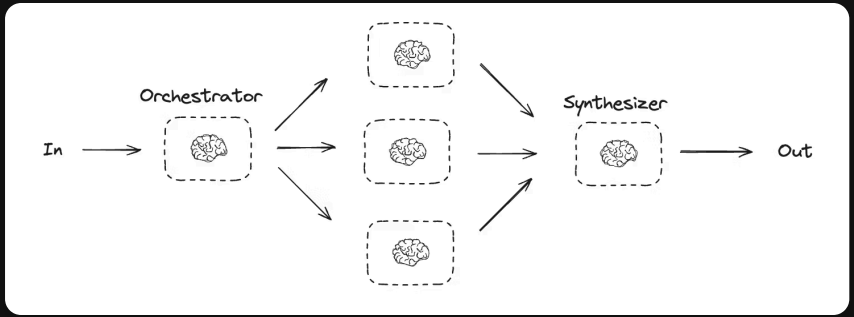

- 오케스트레이터가 **태스크를 분해**

# Orchestrator-Worker



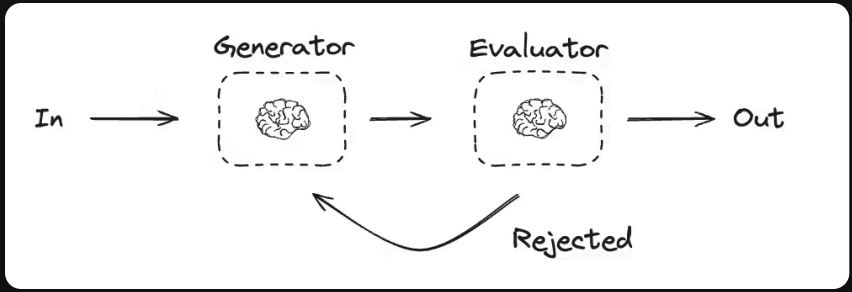

- 오케스트레이터가 **태스크를 분해**
- 워커에게 위임 후 결과 통합

# Orchestrator-Worker

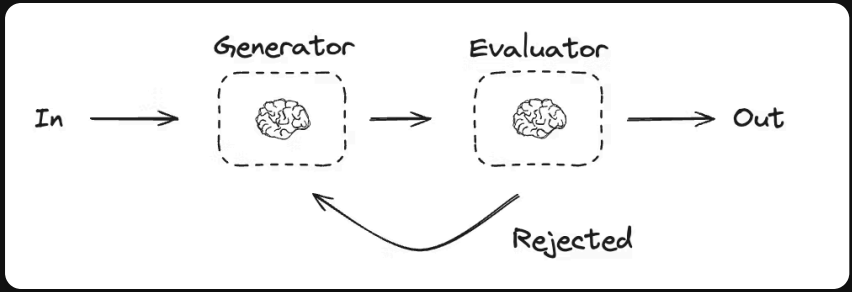



- 오케스트레이터가 **태스크를 분해**
- 워커에게 위임 후 결과 통합
- 예: 리서치 계획 → 병렬 조사 → 보고서

# Evaluator-Optimizer

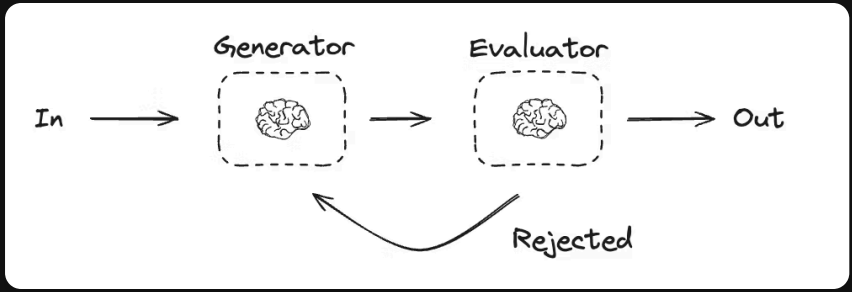


# Evaluator-Optimizer



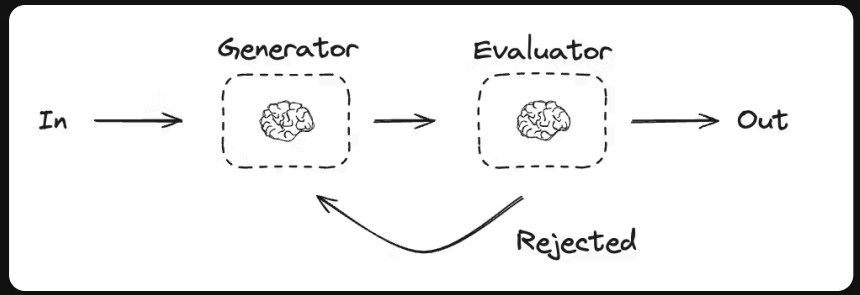

- 생성 LLM + 평가 LLM **반복 개선**

# Evaluator-Optimizer



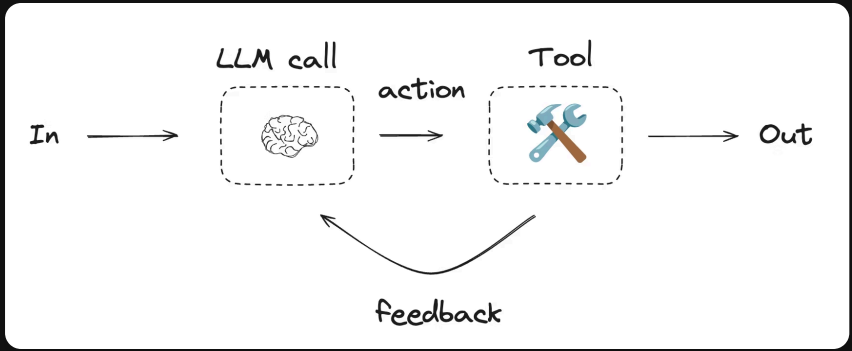

- 생성 LLM + 평가 LLM **반복 개선**
- 품질 기준 달성까지 루프

# Evaluator-Optimizer



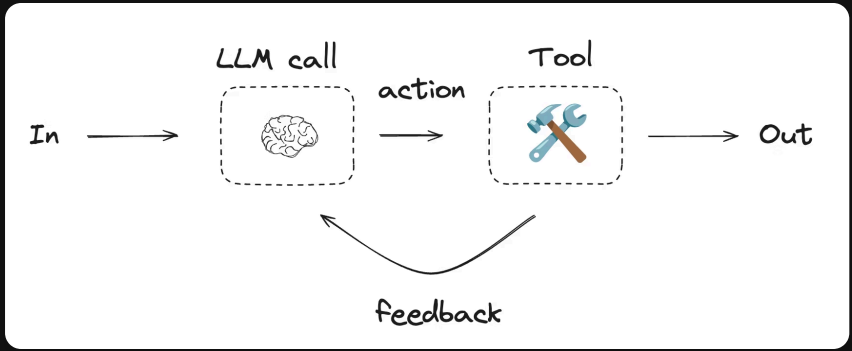

- 생성 LLM + 평가 LLM **반복 개선**
- 품질 기준 달성까지 루프
- 예: 코드 생성 → 테스트 → 수정

# Agent 패턴 — Tool-Calling Loop




출처: docs.langchain.com/oss/python/langgraph/workflows-agents

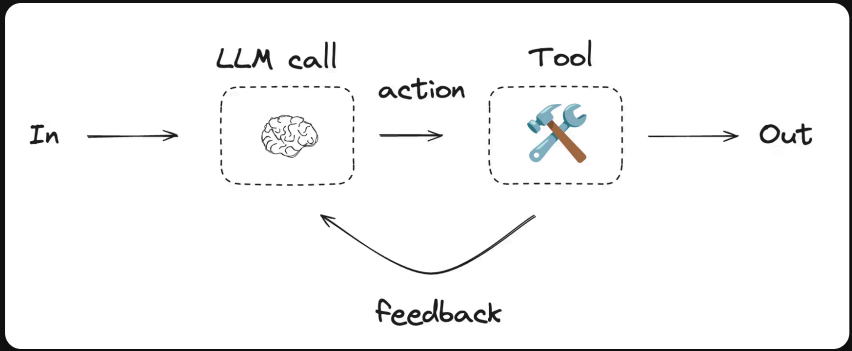
# Agent 패턴 — Tool-Calling Loop




- LLM이 **스스로 도구 선택**과 순서를 결정

출처: docs.langchain.com/oss/python/langgraph/workflows-agents

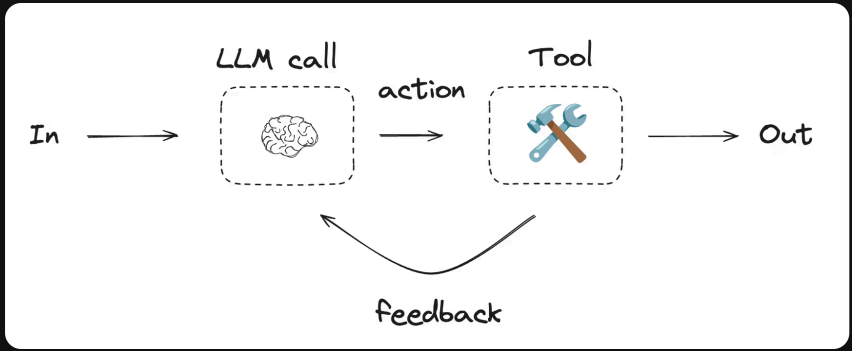
# Agent 패턴 — Tool-Calling Loop




- LLM이 **스스로 도구 선택**과 순서를 결정
- 연속 피드백 루프로 동작

출처: docs.langchain.com/oss/python/langgraph/workflows-agents

# Agent 패턴 — Tool-Calling Loop




- LLM이 **스스로 도구 선택**과 순서를 결정
- 연속 피드백 루프로 동작
- `create_react_agent` 가 이 패턴을 구현

출처: docs.langchain.com/oss/python/langgraph/workflows-agents

# 실습: LangGraph 워크플로

**01_langgraph_workflows 노트북**

# 실습: LangGraph 워크플로

**01_langgraph_workflows 노트북**

- `02_graph_api.ipynb` — StateGraph, 조건 분기, 리듀서 (30분)

# 실습: LangGraph 워크플로

**01_langgraph_workflows 노트북**

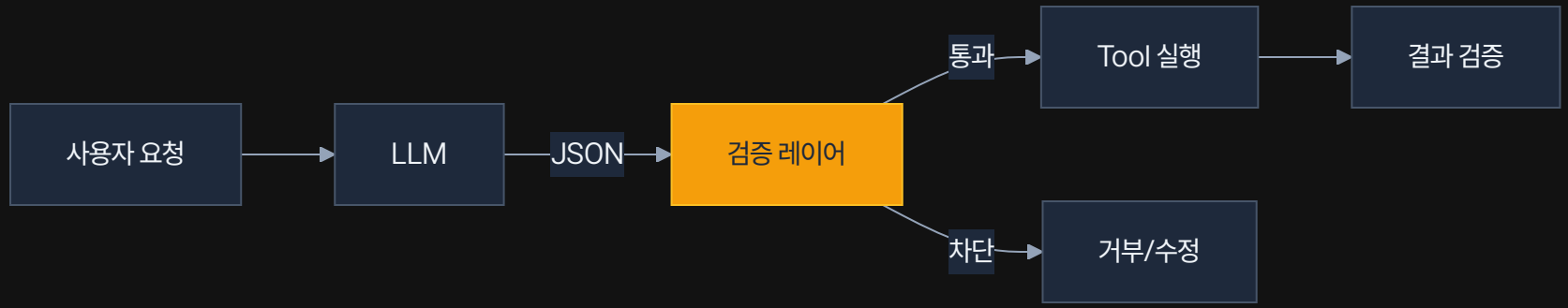
- `02_graph_api.ipynb` — StateGraph, 조건 분기, 리듀서 (30분)
- `04_workflows.ipynb` — 5가지 워크플로 패턴 실행 (40분)

# Tool 호출 통제

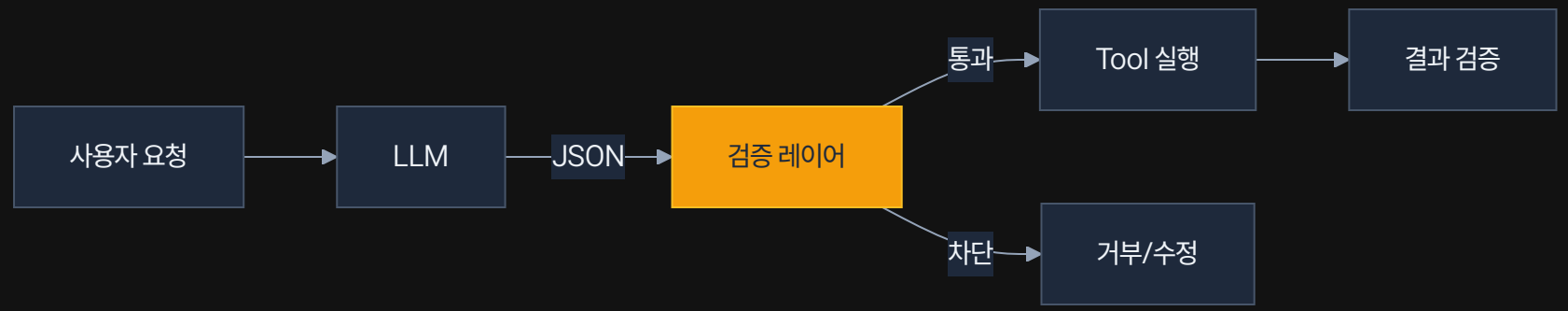
Schema · Validation · Middleware

Session 3 이론 + 실습

# Tool 호출의 위험

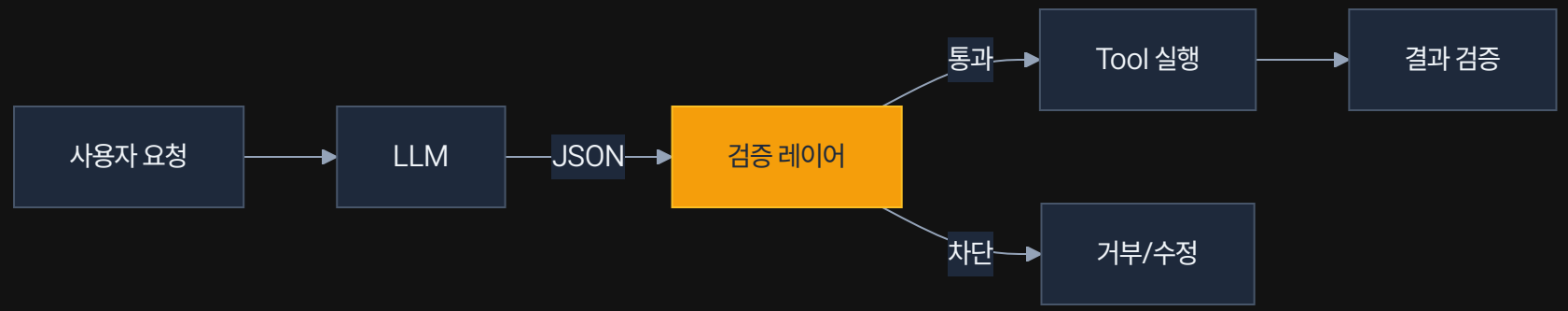


# Tool 호출의 위험



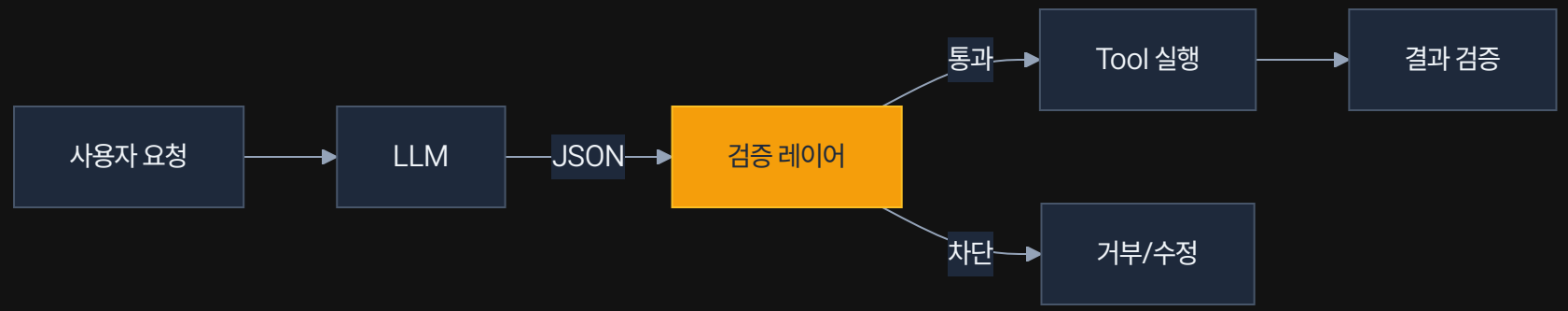

- LLM은 여전히 **잘못된 인자**로 Tool을 호출한다

# Tool 호출의 위험




- LLM은 여전히 **잘못된 인자**로 Tool을 호출한다
- 이메일 발송, 파일 삭제 등 **되돌릴 수 없는 행동**

# Tool 호출의 위험




- LLM은 여전히 **잘못된 인자**로 Tool을 호출한다
- 이메일 발송, 파일 삭제 등 **되돌릴 수 없는 행동**
- 반드시 **검증 레이어**를 거쳐야 한다

# Pydantic args_schema

```python
from pydantic import BaseModel, Field
from langchain_core.tools import tool

class SearchInput(BaseModel):
    query: str = Field(
        description="검색 키워드. 최대 100자.",
        max_length=100,
    )
    max_results: int = Field(
        default=5, ge=1, le=20,
    )

@tool(args_schema=SearchInput)
def web_search(query: str, max_results: int = 5):
    """웹에서 정보를 검색합니다.
    실시간 뉴스, 특정 사실 확인에 사용하세요.
    LLM이 이미 아는 일반 지식에는 사용하지 마세요."""
```

# Pydantic args_schema

```python
from pydantic import BaseModel, Field
from langchain_core.tools import tool

class SearchInput(BaseModel):
    query: str = Field(
        description="검색 키워드. 최대 100자.",
        max_length=100,
    )
    max_results: int = Field(
        default=5, ge=1, le=20,
    )

@tool(args_schema=SearchInput)
def web_search(query: str, max_results: int = 5):
    """웹에서 정보를 검색합니다.
    실시간 뉴스, 특정 사실 확인에 사용하세요.
    LLM이 이미 아는 일반 지식에는 사용하지 마세요."""
```

# Pydantic args_schema

```python
from pydantic import BaseModel, Field
from langchain_core.tools import tool

class SearchInput(BaseModel):
    query: str = Field(
        description="검색 키워드. 최대 100자.",
        max_length=100,
    )
    max_results: int = Field(
        default=5, ge=1, le=20,
    )

@tool(args_schema=SearchInput)
def web_search(query: str, max_results: int = 5):
    """웹에서 정보를 검색합니다.
    실시간 뉴스, 특정 사실 확인에 사용하세요.
    LLM이 이미 아는 일반 지식에는 사용하지 마세요."""
```

# 무한 루프 방지 — 3가지 패턴

# 무한 루프 방지 — 3가지 패턴

- **패턴 1**: 동일 Tool 반복 → `search("AI") → 결과 없음 → search("AI") → ...`

# 무한 루프 방지 — 3가지 패턴

- **패턴 1**: 동일 Tool 반복 → `search("AI") → 결과 없음 → search("AI") → ...`
- **패턴 2**: Tool 간 순환 → `analyze() → fetch() → analyze() → ...`

# 무한 루프 방지 — 3가지 패턴

- **패턴 1**: 동일 Tool 반복 → `search("AI") → 결과 없음 → search("AI") → ...`
- **패턴 2**: Tool 간 순환 → `analyze() → fetch() → analyze() → ...`
- **패턴 3**: 품질 미달 루프 → `write() → check() → rewrite() → ...`

# 무한 루프 방지 — 3가지 패턴

- **패턴 1**: 동일 Tool 반복 → `search("AI") → 결과 없음 → search("AI") → ...`
- **패턴 2**: Tool 간 순환 → `analyze() → fetch() → analyze() → ...`
- **패턴 3**: 품질 미달 루프 → `write() → check() → rewrite() → ...`

```python
class LoopGuard:
    def __init__(self, max_steps=20, max_same_tool=3):
        self.max_steps = max_steps
        self.max_same_tool = max_same_tool
    # 최대 스텝 초과 → 강제 중단
    # 동일 Tool 연속 N회 → 강제 중단
```

# 미들웨어로 Tool 통제

```python
# 호출 횟수 제한
from deepagents.middleware import ToolCallLimitMiddleware

middleware = ToolCallLimitMiddleware(max_calls=20)

# 커스텀: 실행 시간 측정
@wrap_tool_call
def timing_middleware(tool_call, next):
    start = time.time()
    result = next(tool_call)
    print(f"{tool_call.name}: {time.time()-start:.2f}s")
    return result
```

# 미들웨어로 Tool 통제

```python
# 호출 횟수 제한
from deepagents.middleware import ToolCallLimitMiddleware

middleware = ToolCallLimitMiddleware(max_calls=20)

# 커스텀: 실행 시간 측정
@wrap_tool_call
def timing_middleware(tool_call, next):
    start = time.time()
    result = next(tool_call)
    print(f"{tool_call.name}: {time.time()-start:.2f}s")
    return result
```

# 미들웨어로 Tool 통제

```python
# 호출 횟수 제한
from deepagents.middleware import ToolCallLimitMiddleware

middleware = ToolCallLimitMiddleware(max_calls=20)

# 커스텀: 실행 시간 측정
@wrap_tool_call
def timing_middleware(tool_call, next):
    start = time.time()
    result = next(tool_call)
    print(f"{tool_call.name}: {time.time()-start:.2f}s")
    return result
```

미들웨어 = Tool 호출 전후에 **코드를 끼워넣는** 패턴

# 실습: Tool 검증 & 미들웨어

**02_tool_middleware 노트북**

# 실습: Tool 검증 & 미들웨어

**02_tool_middleware 노트북**

- `04_tools.ipynb` — Pydantic 스키마, Structured Output (25분)

# 실습: Tool 검증 & 미들웨어

**02_tool_middleware 노트북**

- `04_tools.ipynb` — Pydantic 스키마, Structured Output (25분)
- `01_middleware.ipynb` — ToolCallLimit, 커스텀 미들웨어 (25분)

# 구조 리팩토링

단일 Agent → 확장형 설계

Session 4 이론

# 단일 Agent의 문제

```python
def run_agent(input):
    result1 = llm_call_1(input)        # LLM과 로직이 섞임
    result2 = tool_call(result1)       # Tool이 직접 연결됨
    result3 = llm_call_2(result2)      # 중간 상태 추적 불가
    return result3
```

# 단일 Agent의 문제

```python
def run_agent(input):
    result1 = llm_call_1(input)        # LLM과 로직이 섞임
    result2 = tool_call(result1)       # Tool이 직접 연결됨
    result3 = llm_call_2(result2)      # 중간 상태 추적 불가
    return result3
```

[확장형 구조]

```
Planner  →  Executor  →  Validator
   ↑           ↑           ↑
교체 가능   독립 테스트   독립 교체
```

# 전환 시점 판단

# 전환 시점 판단

- Tool이 **3개를 초과**할 때

# 전환 시점 판단

- Tool이 **3개를 초과**할 때
- 분기 조건이 **2개를 초과**할 때

# 전환 시점 판단

- Tool이 **3개를 초과**할 때
- 분기 조건이 **2개를 초과**할 때
- 실행 시간이 **10초를 초과**할 때

# 전환 시점 판단

- Tool이 **3개를 초과**할 때
- 분기 조건이 **2개를 초과**할 때
- 실행 시간이 **10초를 초과**할 때
- 팀원이 코드를 혼자 **이해하기 어렵다**고 할 때

# 전환 시점 판단

- Tool이 **3개를 초과**할 때
- 분기 조건이 **2개를 초과**할 때
- 실행 시간이 **10초를 초과**할 때
- 팀원이 코드를 혼자 **이해하기 어렵다**고 할 때

"조기 최적화를 피한다" — 단순하게 시작하고, 병목이 **증명**되었을 때 확장

# Observability — Langfuse 연동

```
from langfuse.langchain import CallbackHandler
handler = CallbackHandler()

result = agent.invoke(
    {"messages": [...]},
    config={"callbacks": [handler]},
)
# → Langfuse 대시보드에서 Trace 확인
```

# Observability — Langfuse 연동

```python
from langfuse.langchain import CallbackHandler
handler = CallbackHandler()

result = agent.invoke(
    {"messages": [...]},
    config={"callbacks": [handler]},
)
# → Langfuse 대시보드에서 Trace 확인
```

# Langfuse가 추적하는 것

# Langfuse가 추적하는 것

- Node별 **실행 시간** — 어디서 병목이 발생하는가

# Langfuse가 추적하는 것

- Node별 **실행 시간** — 어디서 병목이 발생하는가
- **토큰 비용** — 호출당 얼마나 소비하는가

# Langfuse가 추적하는 것

- Node별 **실행 시간** — 어디서 병목이 발생하는가
- **토큰 비용** — 호출당 얼마나 소비하는가
- **오류 로그** — 어떤 Node에서 실패했는가

# Langfuse가 추적하는 것

- Node별 **실행 시간** — 어디서 병목이 발생하는가
- **토큰 비용** — 호출당 얼마나 소비하는가
- **오류 로그** — 어떤 Node에서 실패했는가
- LangSmith 또는 Langfuse 중 선택 가능

# Langfuse가 추적하는 것

- Node별 **실행 시간** — 어디서 병목이 발생하는가
- **토큰 비용** — 호출당 얼마나 소비하는가
- **오류 로그** — 어떤 Node에서 실패했는가
- LangSmith 또는 Langfuse 중 선택 가능

Trace 없이 프로덕션에 배포하지 않는다 — **유일한 디버깅 수단**

# Deep Agents가 자동화하는 것

| 수동 구현 | Deep Agents 자동화 |
| --- | --- |
| Planner 클래스 | **write_todos** |
| Executor 분리 | **task** (서브에이전트) |
| Tool 래퍼 | **FilesystemMiddleware** |
| Memory 관리 | **SummarizationMiddleware** |
| HITL 게이트 | **interrupt_on** |

# Deep Agents가 자동화하는 것

| 수동 구현 | Deep Agents 자동화 |
|---|---|
| Planner 클래스 | **write_todos** |
| Executor 분리 | **task** (서브에이전트) |
| Tool 래퍼 | **FilesystemMiddleware** |
| Memory 관리 | **SummarizationMiddleware** |
| HITL 게이트 | **interrupt_on** |

```
agent = create_deep_agent(
    model=model,
    interrupt_on={"execute": True},  # 위험한 도구는 승인 필요
)
```

# Day 2 핵심 정리

**Agent 4요소**

Goal · Memory · Tool · Control Logic

**프레임워크 계층**

LangChain → LangGraph → Deep Agents

**LangGraph 핵심**

Node-Edge-State + Conditional Edge

**Tool 통제**

Schema + Middleware + LoopGuard

# Day 3 예고

## MCP · RAG 구현 & 외부 시스템 연동

MCP 고급 설계 · RAG 튜닝 · Hybrid 아키텍처